ECSE 420 Parallel Computing, Assignment 1, Fall 2026

Student 1 name: Roman Humenyuk

Student 1 ID: 261240585

Student 2 name: Ziang Wang

Student 2 ID: 261161981

How to run each file and which question each one solves

All source files are in the package `ca.mcgill.ecse420.a1`. From the source root (the folder that contains `ca/`), compile with:

```
javac ca/mcgill/ecse420/a1/*.java
```

then run each part with the command shown below (from the same folder):

- `java ca.mcgill.ecse420.a1.MatrixMultiplication validate` → Q1.1, Q1.2 (correctness checks)
- `java ca.mcgill.ecse420.a1.MatrixMultiplication` → Q1.3 (one timed run, 2000×2000)
- `java ca.mcgill.ecse420.a1.MatrixMultiplication threads 4000` → Q1.4
- `java ca.mcgill.ecse420.a1.MatrixMultiplication sizes <t*>` → Q1.5 (t* = best thread count from Q1.4)
- `java ca.mcgill.ecse420.a1.Deadlock` → Q2.1
- `java ca.mcgill.ecse420.a1.Deadlock fixed` → Q2.2
- `java ca.mcgill.ecse420.a1.DiningPhilosophers deadlock 8` → Q3.1
- `java ca.mcgill.ecse420.a1.DiningPhilosophers nodeadlock 8` → Q3.2 (deadlock-free version)
- `java ca.mcgill.ecse420.a1.DiningPhilosophers nostarvation 8` → Q3.2 (starvation-free version)

For DiningPhilosophers the optional arguments are `[mode] [seconds] [philosophers]` (defaults: nostarvation, 8 s, 5 philosophers). No input files are needed.

Question 1: Matrix multiplication

1.1 `sequentialMultiplyMatrix` (Appendix A, `MatrixMultiplication.java`) computes the matrix product: for an n×m matrix a and an m×p matrix b, the result is the n×p matrix with result[i][j] = Σ a[i][k]·b[k][j] (sum over k = 0 … m − 1).

The result is computed one row at a time by a helper method, `multiplyRow`. For row i, it goes through k = 0 … m − 1, multiplies row k of b by a[i][k] and adds it to row i of the result. This is the i-k-j loop order. It gives exactly the same values as the textbook i-j-k order (dot product of row i of a with column j of b), but it reads b row by row instead of column by column, which uses the CPU cache much better. Our first version used the i-j-k order and was about 9× slower on large matrices (see 1.5).

Validation. The method is checked by the `validate()` method, run with `java ca.mcgill.ecse420.a1.MatrixMultiplication validate`:

1. Hand-computed cases: a 2×2 product ([[1,2],[3,4]]·[[5,6],[7,8]] = [[19,22],[43,50]]) and a non-square 2×3 · 3×2 product ([[1,2,3],[4,5,6]]·[[7,8],[9,10],[11,12]] = [[58,64],[139,154]]), whose expected values were calculated by hand.
2. Algebraic identities on a random 50×50 matrix M: M·I = M and M·0 = 0.
3. Shape check: a 301×200 by 200×157 product has 301 rows and 157 columns.

All checks passed. The full console output (which also covers the parallel checks of 1.2) is:

```
PASS  2x2 hand-computed (sequential)
PASS  2x2 hand-computed (parallel, 2 threads)
PASS  2x3 * 3x2 hand-computed (sequential)
PASS  2x3 * 3x2 hand-computed (parallel, 2 threads)
PASS  M * I = M (sequential)
PASS  M * I = M (parallel, 4 threads)
PASS  M * 0 = 0 (sequential)
PASS  M * 0 = 0 (parallel, 4 threads)
PASS  301x200 * 200x157 gives a 301x157 result
PASS  parallel (1 threads) equals sequential
PASS  parallel (2 threads) equals sequential
PASS  parallel (3 threads) equals sequential
PASS  parallel (7 threads) equals sequential
PASS  parallel (16 threads) equals sequential
PASS  parallel (500 threads) equals sequential
ALL CHECKS PASSED
```

1.2 *Task definition.* `parallelMultiplyMatrix(a, b, threads)` splits the rows of the result matrix into `threads` contiguous blocks. With n rows and t threads, each block has ⌊n/t⌋ rows, and the first n mod t blocks get one extra row, so every row is covered exactly once. The thread count is capped at the number of rows so that no thread gets an empty block. Each block becomes one

`Callable` task that computes its rows with the same `multiplyRow` helper as the sequential version. The tasks run on a fixed-size thread pool (`Executors.newFixedThreadPool`). The two-argument `parallelMultiplyMatrix(a, b)` required by the assignment calls this method with `NUMBER_THREADS` (32).

Combining results. All tasks read the input matrices a and b (never modified) and write into one shared result array. Because the row ranges don't overlap, no two tasks ever write the same cell, so no locks are needed: the result is combined just by each task filling in its own rows of the same array. The main thread submits the tasks with `invokeAll`, which waits until every task has finished, and then calls `get()` on each returned `Future`. By the `ExecutorService` memory-consistency guarantee, everything a task does happens-before `Future.get()` returns, so all writes made by the worker threads are visible to the main thread when the result is returned. `get()` also rethrows any exception from a task, so a failed task can't silently leave a half-filled matrix. The pool is shut down in a `finally` block.

Validation. Besides the hand-computed and identity cases from 1.1 (also run through the parallel method), the main test compares the parallel result with the sequential result on a random non-square 301×200 by 200×157 product, using 1, 2, 3, 7, 16 and 500 threads. 3 and 7 test the uneven split (301 is not divisible by 3 or 7), and 500 > 301 tests the cap on the number of threads. Every parallel result was identical to the sequential result (see the output in 1.1), which confirms both the row splitting and that there are no race conditions.

1.3 `measureExecutionTime(a, b)` records `System.nanoTime()` right before and after each of the two multiplications and prints the elapsed wall-clock time in milliseconds for the sequential and parallel methods. Output of `java ca.mcgill.ecse420.a1.MatrixMultiplication`:

```
Matrix size 2000x2000, 32 thread(s)

  Sequential: 2391.37 ms

  Parallel:   515.57 ms
```

For the experiments in 1.4 and 1.5, `averageTimeMs` uses the same idea with two improvements: one untimed warm-up run first, so JIT compilation and class loading don't slow down the first measurement, and then the average of `REPETITIONS` = 3 timed runs to smooth out noise.

Plausibility tests. We checked that the timings make sense:

- Scaling: the algorithm does n^3 multiply-adds, so doubling n should multiply the time by about 8. From the 1.5 table, sequential 500×500 took 20.39 ms and 1000×1000 took 171.34 ms, a ratio of 8.4 ≈ 8.
- Speed-up: with p threads on c cores, the parallel time should be at best the sequential time divided by min(p, c). With 32 threads on 1000×1000 (8 cores) we measured a speed-up of 4.69×, which is below the limit of 8, as expected.

- Repeatability: single runs vary at large sizes. The single 2000×2000 sequential run above took 2391 ms, while the average in 1.5 was 4439 ms. This is why 1.4 and 1.5 use a warm-up plus an average of 3 runs.
- Sanity: 100×100 matrices take under 2 ms, while 4000×4000 takes about 42.7 s sequentially, consistent with n^3 growth.

1.4 Test machine: Apple M1 Pro (8 cores: 6 performance + 2 efficiency, 8 logical processors), 16 GB RAM, OpenJDK 17.0.16, macOS 26.5. Command: `java ca.mcgill.ecse420.a1.MatrixMultiplication threads 4000`. Results (4000×4000, average of 3 runs after one warm-up run):

```
threads,time_ms,speedup
1,36521.28,1.00
2,28618.40,1.28
4,18695.87,1.95
8,17116.59,2.13
16,16622.54,2.20
32,16559.49,2.21
```

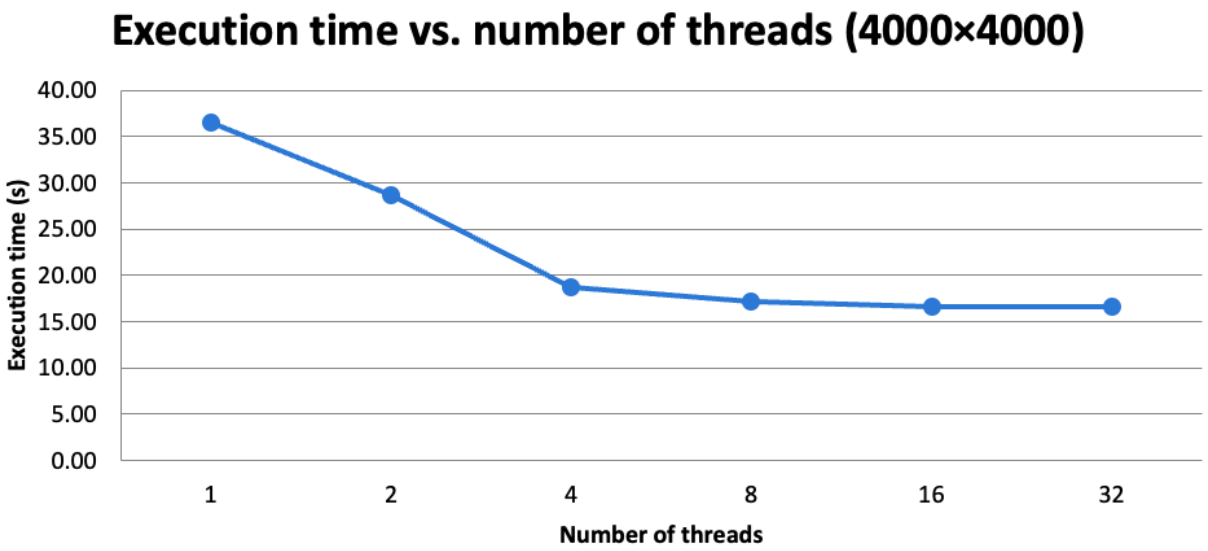


Figure 1: Execution time vs. number of threads (4000×4000).

Discussion. The time drops from 36.5 s with 1 thread to 18.7 s with 4 threads (1.95× faster). After that the curve is almost flat: 8, 16 and 32 threads all take about 16.6–17.1 s, and the best speed-up is only 2.21× with 32 threads, even though the machine has 8 cores.

The main reason is memory, not computation. To compute one row of the result, a thread reads all of matrix b, which is 4000 × 4000 doubles = 128 MB, much bigger than the CPU caches. So

every thread keeps streaming b from main memory, and all threads share the same memory bandwidth. Once about 4 threads are running, memory is the bottleneck and extra threads mostly wait for data. With 16 and 32 threads there are more threads than cores, so they just take turns on the same 8 cores; this adds a bit of scheduling and context-switching overhead but no real speed-up. The M1 Pro also has only 6 fast performance cores (the other 2 are slower efficiency cores), so even without the memory limit the speed-up could not reach 8×. Finally, Amdahl's law applies: the sequential parts (allocating the result, creating the thread pool, waiting for the tasks) put a limit on the speed-up.

The minimum time was obtained with 32 threads (16.56 s), so 32 is the value used in 1.5 and set as `NUMBER_THREADS`. 8, 16 and 32 threads are within about 3% of each other.

1.5 Command: `java ca.mcgill.ecse420.a1.MatrixMultiplication sizes 32`. Results (average of 3 runs after one warm-up run):

```
size,sequential_ms,parallel_ms,speedup
100,1.74,2.41,0.72
200,1.41,2.17,0.65
500,20.39,6.58,3.10
1000,171.34,36.53,4.69
2000,4438.59,2269.00,1.96
3000,16632.07,6735.93,2.47
4000,42690.75,23378.41,1.83
```

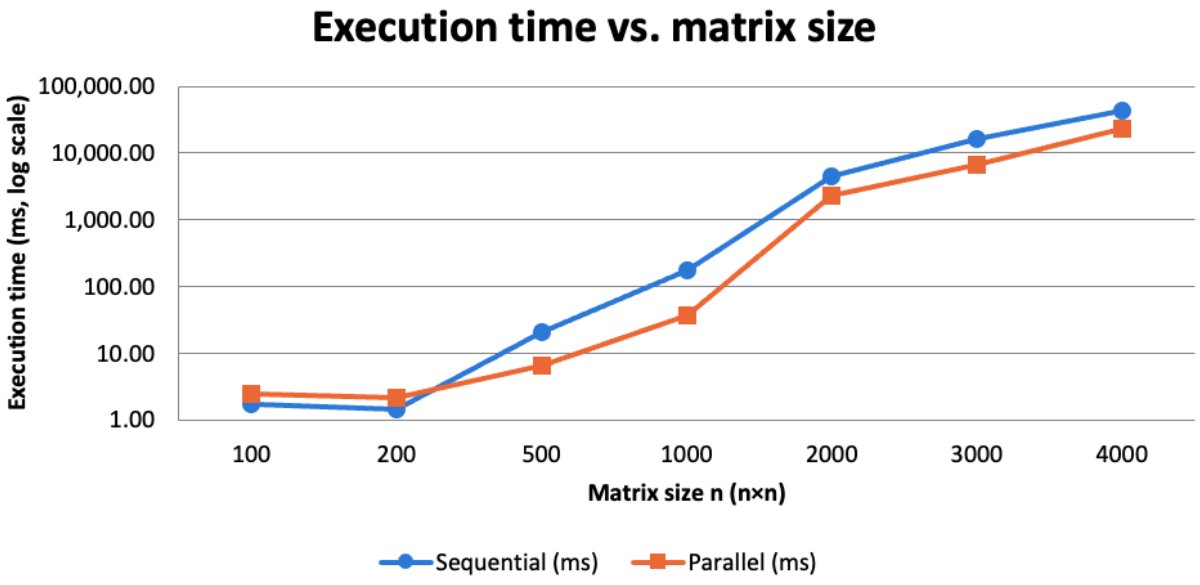


Figure 2: Execution time vs. matrix size, sequential and parallel (32 threads). Both axes use a log scale because the times range from about 1 ms to 43,000 ms.

Discussion. Both curves grow quickly with the size, as expected from the n^3 number of operations. On the log-log graph this shows up as lines that get steeper as n grows.

For the smallest matrices (100×100 and 200×200) the parallel version is actually slower (speed-up 0.72× and 0.65×). The multiplication only takes about 2 ms, so the fixed cost of creating a pool of 32 threads and handing out the tasks is bigger than the work itself.

For 500×500 and 1000×1000 the parallel version is clearly faster, with the best speed-up of 4.69× at 1000. At this size matrix b is 1000 × 1000 doubles = 8 MB, which still fits in the CPU cache, so the threads are mostly limited by how fast they can compute.

Between 1000 and 2000 the sequential time jumps by about 26× (171 ms → 4439 ms), much more than the 8× expected from n^3. At 2000×2000, b is 32 MB and no longer fits in the cache, so it has to be read from main memory over and over. From 2000 up, the speed-up drops to about 2× (1.96×, 2.47×, 1.83×), the same as in 1.4: the threads all compete for the same memory bandwidth.

The loop order also matters a lot for the same reason. Our first version used the textbook i-j-k order, which reads b down a column and jumps to a different row of memory on every step. It took 331 s with one thread on 4000×4000. The i-k-j version reads b row by row and took 36.5 s, about 9× faster for exactly the same arithmetic.

Question 2: Deadlock

2.1 Deadlock is a situation where a set of threads can never make progress, because each one is waiting for a resource that is held by another thread in the same set. Four conditions must all hold at the same time for deadlock to happen (the Coffman conditions):

1. *Mutual exclusion* – a resource can be held by only one thread at a time.
2. *Hold and wait* – a thread holds at least one resource while waiting to get another.
3. *No preemption* – a resource can't be taken away from the thread holding it; it has to be released voluntarily.
4. *Circular wait* – there is a cycle of threads T1 → T2 → … → Tn → T1 where each thread waits for a resource held by the next.

Consequences: the deadlocked threads block forever, so their work is never finished. Any resources they hold (locks, memory, connections) stay locked, which can block other threads that later need them too. Deadlock doesn't throw an exception; the program just looks like it hangs, which makes it hard to detect and often means the application has to be restarted.

Program. In `Deadlock.java` (Appendix A), two `ReentrantLock` objects, `lockA` and `lockB`, are shared by two threads. Thread 1 gets `lockA` first and then asks for `lockB`; Thread 2 gets `lockB` first and then asks for `lockA` (both through the method `acquireBothLocks`). A 100 ms sleep after the first lock makes it practically certain (but not strictly guaranteed) that both threads have their first lock before either asks for its second one. All four conditions then hold: a `ReentrantLock` can be held by one thread at a time (mutual exclusion); each thread holds one lock while waiting for the other (hold and wait); Java never takes a lock away from its owner (no preemption); and Thread 1 waits for a lock held by Thread 2 while Thread 2 waits for a lock held by Thread 1 (circular wait).

So that the program ends on its own instead of hanging, the two threads are daemon threads and main waits for them for at most 2 s. If they haven't finished by then, it asks the JVM's deadlock detector (`ThreadMXBean.findDeadlockedThreads()`) which threads are deadlocked and prints them. Output of `java ca.mcgill.ecse420.a1.Deadlock`:

```
Thread 2: acquired lock B

Thread 1: acquired lock A

Thread 2: waiting for lock A while holding lock B

Thread 1: waiting for lock B while holding lock A

DEADLOCK detected after 2000 ms:

  Thread 1 is blocked on a lock owned by Thread 2

  Thread 2 is blocked on a lock owned by Thread 1

"Both threads finished." was never printed.
```

Both threads get their first lock and say they are waiting for the second; after that nothing happens, the JVM confirms that each thread is blocked on the lock owned by the other, and "Both threads finished." is never printed.

2.2 Since deadlock needs all four conditions, it can be prevented by breaking any one of them:

- *Lock ordering (breaks circular wait)* – give all locks a global order and make every thread take locks in that order. A cycle then becomes impossible, because a thread holding a "later" lock can never wait for an "earlier" one.
- *Try-lock with timeout or back-off (breaks hold and wait / no preemption)* – use `tryLock()`; if the second lock can't be obtained, release the first lock, wait, and retry. This avoids blocking forever, but it can cause livelock without a random back-off.
- *Acquire all resources at once (breaks hold and wait)* – request every lock a task needs in one atomic step, or use one bigger lock that protects both resources. Simpler, but less concurrency.

- *Avoid nested locking* – restructure the code so a thread never holds one lock while asking for another, e.g. by using higher-level concurrent data structures or message passing instead of shared locks.
- *Deadlock detection and recovery* – let deadlock happen but detect it (e.g. with `ThreadMXBean.findDeadlockedThreads()`) and abort or restart one of the threads. Used in databases, but usually a last resort.

Yes, the first solution applies directly to our program, because the deadlock only exists since the two threads take the locks in opposite orders. Running `java ca.mcgill.ecse420.a1.Deadlock fixed` makes Thread 2 call `acquireBothLocks("Thread 2", lockA, "A", lockB, "B")`, so both threads take `lockA` before `lockB`. Whichever thread gets `lockA` first can also get `lockB`, finishes, and releases both; then the other thread continues. Circular wait can no longer happen:

```
Thread 1: acquired lock A

Thread 1: waiting for lock B while holding lock A

Thread 1: acquired lock B

Thread 2: acquired lock A

Thread 2: waiting for lock B while holding lock A

Thread 2: acquired lock B

Both threads finished.
```

The try-lock approach would also work here, but lock ordering is the simplest fix because it adds no extra code paths and no retry logic.

Question 3: Dining philosophers

3.1 In `DiningPhilosophers.java` (Appendix A), each philosopher is a `Runnable` run by its own thread in a fixed thread pool, and each chopstick is a `ReentrantLock` shared by the two neighbouring philosophers: philosopher i uses chopstick i (left) and chopstick (i + 1) mod n (right). The number of philosophers n is a command-line argument (5 by default). A philosopher thinks for a random time, picks up both chopsticks, eats for a random time, puts them down, and repeats. The program runs for a fixed time and then prints how many meals each philosopher ate.

Picking up a chopstick means acquiring its lock. Since a lock can only be held by one thread at a time, two philosophers can never hold the same chopstick at the same time. Chopsticks are always put down in `finally` blocks, so a chopstick is always released.

In `deadlock` mode, every philosopher picks up the left chopstick first, then the right one, and holds the left one while waiting for the right one. If all n philosophers pick up their left chopstick at about the same time, all n chopsticks are taken and each philosopher i waits for chopstick (i +

1) mod n, which philosopher (i + 1) mod n is holding. Nobody can continue and nobody puts anything down, so the philosophers are stuck forever. This is a circular wait, so the philosophers will eventually be deadlocked.

While the simulation runs, main checks every 100 ms with `ThreadMXBean.findDeadlockedThreads()` whether the philosophers are deadlocked. Output of `java ca.mcgill.ecse420.a1.DiningPhilosophers deadlock 8` (the last chopstick pick-ups before the deadlock, then the summary):

```
Mode: deadlock, 5 philosophers, 8 s, ordered chopsticks: false, fair locks: false
...
Philosopher 1 picked up chopstick 1
...
Philosopher 0 picked up chopstick 0
Philosopher 4 picked up chopstick 4
Philosopher 3 picked up chopstick 3
Philosopher 2 picked up chopstick 2


DEADLOCK detected: every philosopher holds one chopstick and waits for the next one.
Meals eaten:
  Philosopher 0: 152
  Philosopher 1: 145
  Philosopher 2: 149
  Philosopher 3: 149
  Philosopher 4: 141
```

Each philosopher picked up their left chopstick and then nobody ate again: the deadlock happened after about 150 meals per philosopher, even though the program was allowed to run for 8 s.

3.2 *Avoiding deadlock.* We use resource ordering: the chopsticks are numbered 0 to n − 1 and every philosopher picks up their lower-numbered chopstick first (`Math.min(left, right)`, then `Math.max(left, right)`). Only the last philosopher's order changes: they now pick up chopstick 0 before chopstick n − 1. This removes the circular-wait condition. A waiting philosopher always holds a lower-numbered chopstick than the one they are waiting for, so along any chain of waiting philosophers the chopstick numbers keep increasing and the chain can't loop back on

itself. Without a cycle there is no deadlock. Output of `java ca.mcgill.ecse420.a1.DiningPhilosophers nodeadlock 8`:

```
Mode: nodeadlock, 5 philosophers, 8 s, ordered chopsticks: true, fair locks: false
...
No deadlock occurred during the 8 s run.
Meals eaten:
  Philosopher 0: 411
  Philosopher 1: 459
  Philosopher 2: 494
  Philosopher 3: 561
  Philosopher 4: 420
```

Is starvation possible even without deadlock? Yes. Deadlock-freedom only guarantees that some philosopher makes progress, not that every philosopher does. The default `ReentrantLock` is non-fair: when a chopstick is released, a neighbour who just finished eating can grab it again before a philosopher who was already waiting. This can keep happening, so a philosopher can starve.

Avoiding starvation. We amended the program (`nostarvation` mode) so that each chopstick is a fair lock, `new ReentrantLock(true)`, which gives the lock to the thread that has waited the longest. The resource ordering is kept, so it is still deadlock-free. Each chopstick is shared by only two philosophers, so a waiting philosopher gets it as soon as their neighbour puts it down. The neighbour always puts it down eventually, because eating takes a finite time and there is no deadlock. So every hungry philosopher eventually eats.

Testing. Each philosopher counts their meals, and the program prints every count at the end of a fixed-length run. A starving philosopher's count would stay near zero while the others kept growing. Output of `java ca.mcgill.ecse420.a1.DiningPhilosophers nostarvation 8`:

```
Mode: nostarvation, 5 philosophers, 8 s, ordered chopsticks: true, fair locks: true

...

No deadlock occurred during the 8 s run.

Meals eaten:

  Philosopher 0: 413

  Philosopher 1: 463

  Philosopher 2: 496

  Philosopher 3: 551

  Philosopher 4: 450
```

In the 8-second run with 5 philosophers the program never got stuck and the meal counts were 413, 463, 496, 551 and 450, so every philosopher kept eating. Testing gives evidence, not proof: starvation-freedom is guaranteed by the fair-lock argument above.

Question 4: Amdahl's law

Throughout, the program takes unit time when executed sequentially ($T_1 = 1$). If a fraction s of that time is sequential and the remaining 1 − s is perfectly parallelizable, then on n processors T_n = s + (1 − s)/n and Amdahl's law gives the speed-up S_n = T_1/T_n = 1 / (s + (1 − s)/n).

4.1 Here s = 0.3, so the parallel part is 1 − 0.3 = 0.7.

```
T_n = 0.3 + 0.7/n
S_n = 1 / (0.3 + 0.7/n)
    = n / (0.3n + 0.7)            (multiply top and bottom by n)
    = 10n / (3n + 7)              (multiply top and bottom by 10)
```

Check: n = 1 gives S_1 = 10/10 = 1 (no speed-up on one processor), and n = 8 gives S_8 = 80/31 ≈ 2.58.

Limit with an unbounded number of processors: as n → ∞, 0.7/n → 0, so

```
lim S_n = 1 / 0.3 = 10/3 ≈ 3.33   (n → ∞)
```

The 30 % sequential part caps the speed-up at 3.33×, no matter how many processors are added.

4.2 Original program: s = 0.4, so

```
s_n = 1 / (0.4 + 0.6/n)
```

The improved version decreases the sequential time percentage by a factor of k, so the sequential fraction becomes 0.4/k. The program still takes unit time sequentially, so the parallel fraction becomes 1 − 0.4/k:

```
s'_n = 1 / (0.4/k + (1 – 0.4/k)/n)
```

Require s'_n = 2·s_n. Taking reciprocals:

```
0.4/k + (1 − 0.4/k)/n = (0.4 + 0.6/n) / 2 = 0.2 + 0.3/n
0.4n/k + 1 − 0.4/k    = 0.2n + 0.3          (multiply both sides by n)
(0.4/k)(n − 1)        = 0.2n − 0.7
k = 0.4(n − 1) / (0.2n − 0.7)
k = 4(n − 1) / (2n − 7)
```

Validity. k must be positive, so 2n − 7 > 0, i.e. n ≥ 4. For n ≤ 3 the speed-up cannot be doubled at all: even removing the sequential part completely (k → ∞) gives s'_n → n, which is less than 2·s_n (e.g. n = 3: 2·s_3 = 3.33 > 3). As n → ∞, k → 4/2 = 2. Check with n = 4: k = 12; s_4 = 1/(0.4 + 0.15) = 1.818 and s'_4 = 1/(0.4/12 + (1 − 0.4/12)/4) = 1/0.275 = 3.636 = 2·s_4 ✓.

4.3 Let p be the fraction of the (unit) sequential execution time taken by the sequential part. On n processors:

```
Original:  T_n  = p + (1 − p)/n
Modified:  T'_n = p/3 + (1 − p/3)/n      (sequential percentage decreased 3-fold)
```

Require T'_n = T_n / 2:

```
p/3 + (1 − p/3)/n = p/2 + (1 − p)/(2n)
2pn + 6 − 2p      = 3pn + 3 − 3p          (multiply both sides by 6n)
3 + p             = pn
p(n − 1)          = 3
p = 3 / (n − 1)
```

Validity. p is a fraction, so p ≤ 1, which requires n ≥ 4. Check with n = 7: p = 0.5; T_7 = 0.5 + 0.5/7 = 0.5714 and T'_7 = 0.5/3 + (1 − 0.5/3)/7 = 0.2857 = T_7/2 ✓.

Appendix A – Source Code

```java
package ca.mcgill.ecse420.a1;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class MatrixMultiplication {

  // Number of threads used by parallelMultiplyMatrix(a, b).
  // Set this to the best thread count found in Question 1.4.
  private static final int NUMBER_THREADS = 32;
  private static final int MATRIX_SIZE = 2000;
  private static final int REPETITIONS = 3; // timed runs averaged per measurement
```

```java
  public static void main(String[] args) {
    String mode = args.length > 0 ? args[0] : "";
    switch (mode) {
      case "validate" -> {
        boolean allPassed = validate();
        System.out.println(allPassed ? "ALL CHECKS PASSED" : "SOME CHECKS FAILED");
      }
      case "threads" -> {
        int matrixSize = args.length > 1 ? Integer.parseInt(args[1]) : 4000;
        threadCountExperiment(matrixSize);
      }
      case "sizes" -> {
        int threadCount = args.length > 1 ? Integer.parseInt(args[1]) :
NUMBER_THREADS;
        matrixSizeExperiment(threadCount);
      }
      default -> {
        // Generate two random matrices, same size
        double[][] a = generateRandomMatrix(MATRIX_SIZE, MATRIX_SIZE);
        double[][] b = generateRandomMatrix(MATRIX_SIZE, MATRIX_SIZE);
        measureExecutionTime(a, b);
      }
    }
  }

  /**
   * Returns the result of a sequential matrix multiplication
   * The two matrices are randomly generated
   * @param a is the first matrix
   * @param b is the second matrix
   * @return the result of the multiplication
   * */
  public static double[][] sequentialMultiplyMatrix(double[][] a, double[][] b) {
    int numRows = a.length; // rows of a -> rows of the result
    int numCols = b[0].length; // columns of b -> columns of the result
    double[][] result = new double[numRows][numCols];

    // result[i][j] = sum over k of a[i][k] * b[k][j], computed one row of the result
at a time
    for (int i = 0; i < numRows; i++) {
      multiplyRow(a, b, result, i);
    }
    return result;
  }

  /**
   * Returns the result of a concurrent matrix multiplication
```

```
   * The two matrices are randomly generated
   * @param a is the first matrix
   * @param b is the second matrix
   * @return the result of the multiplication
   * */
  public static double[][] parallelMultiplyMatrix(double[][] a, double[][] b) {
    return parallelMultiplyMatrix(a, b, NUMBER_THREADS);
  }

  /**
   * Same as parallelMultiplyMatrix(a, b) but with an explicit number of threads, so
the thread
   * count can be varied in the experiments.
   *
   * @param a is the first matrix
   * @param b is the second matrix
   * @param requestedThreads number of threads to use
   * @return the result of the multiplication
   */
  public static double[][] parallelMultiplyMatrix(double[][] a, double[][] b,
      int requestedThreads) {
    int numRows = a.length;
    double[][] result = new double[numRows][b[0].length];

    // Never use more threads than there are rows to hand out
    int numThreads = Math.min(requestedThreads, numRows);
    ExecutorService executor = Executors.newFixedThreadPool(numThreads);

    // Split the rows of the result into contiguous blocks, one block per thread.
    // If the rows do not divide evenly, the first (numRows % numThreads) blocks get
one extra row.
    int rowsPerThread = numRows / numThreads;
    int extraRows = numRows % numThreads;
    List<Callable<Void>> tasks = new ArrayList<>();
    int nextRow = 0;
    for (int t = 0; t < numThreads; t++) {
      final int firstRow = nextRow;
      final int endRow = nextRow + rowsPerThread + (t < extraRows ? 1 : 0); //
exclusive
      nextRow = endRow;

      // Each task computes only rows [firstRow, endRow) of the result.
      // Row ranges are disjoint between tasks, so no synchronization on 'result' is
needed.
      tasks.add(() -> {
        for (int i = firstRow; i < endRow; i++) {
          multiplyRow(a, b, result, i);
        }
```

```
        return null;
      });
    }

    try {
      // invokeAll blocks until every task has finished. Calling get() on each Future
      // (1) rethrows any exception thrown inside a task, and (2) guarantees that all
writes
      // made by that task to 'result' are visible to this thread (happens-before).
      for (Future<Void> future : executor.invokeAll(tasks)) {
        future.get();
      }
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      throw new IllegalStateException("Interrupted during parallel multiplication",
e);
    } catch (ExecutionException e) {
      throw new IllegalStateException("A multiplication task failed", e.getCause());
    } finally {
      executor.shutdown();
    }
    return result;
  }

  /**
   * Computes row i of the product a * b and stores it in result[i].
   *
   * <p>The loops are in i-k-j order: for each k, row k of b is multiplied by a[i][k]
and added
   * to row i of the result. This reads b and the result row from left to right in
memory, which
   * uses the CPU cache well. The textbook i-j-k order reads b down a column instead,
jumping to a
   * different row of b on every step, which is several times slower for large
matrices. Both
   * orders compute exactly the same values: result[i][j] = sum over k of a[i][k] *
b[k][j].
   *
   * @param a is the first matrix
   * @param b is the second matrix
   * @param result matrix that receives the product; row i must be all zeros when
called
   * @param i index of the row to compute
   */
  private static void multiplyRow(double[][] a, double[][] b, double[][] result, int
i) {
    double[] resultRow = result[i];
    int innerSize = a[0].length; // columns of a = rows of b
```

```
    for (int k = 0; k < innerSize; k++) {
      double aik = a[i][k];
      double[] bRow = b[k];
      for (int j = 0; j < resultRow.length; j++) {
        resultRow[j] += aik * bRow[j];
      }
    }
  }

  /**
   * Populates a matrix of given size with randomly generated integers between 0-9.
   * @param numRows number of rows
   * @param numCols number of cols
   * @return matrix
   */
  private static double[][] generateRandomMatrix(int numRows, int numCols) {
    double[][] matrix = new double[numRows][numCols];
    for (int row = 0; row < numRows; row++) {
      for (int col = 0; col < numCols; col++) {
        matrix[row][col] = (double) ((int) (Math.random() * 10.0));
      }
    }
    return matrix;
  }

  /**
   * Measures and prints the execution time of the sequential and the parallel matrix
   * multiplication of a and b.
   *
   * @param a is the first matrix
   * @param b is the second matrix
   * @return array of two values: {sequential time, parallel time} in milliseconds
   */
  public static double[] measureExecutionTime(double[][] a, double[][] b) {
    long startTime = System.nanoTime();
    sequentialMultiplyMatrix(a, b);
    double sequentialMs = (System.nanoTime() - startTime) / 1_000_000.0;

    startTime = System.nanoTime();
    parallelMultiplyMatrix(a, b);
    double parallelMs = (System.nanoTime() - startTime) / 1_000_000.0;

    System.out.printf("Matrix size %dx%d, %d thread(s)%n", a.length, b[0].length,
        NUMBER_THREADS);
    System.out.printf("  Sequential: %.2f ms%n", sequentialMs);
    System.out.printf("  Parallel:   %.2f ms%n", parallelMs);
    return new double[] {sequentialMs, parallelMs};
  }
```

```java
  /**
   * Question 1.4: multiplies two size x size matrices in parallel with 1, 2, 4, 8, 16
and 32
   * threads and prints the average execution time and speed-up for each thread count.
   *
   * @param size number of rows and columns of the matrices
   */
  public static void threadCountExperiment(int size) {
    int[] threadCounts = {1, 2, 4, 8, 16, 32};
    double[][] a = generateRandomMatrix(size, size);
    double[][] b = generateRandomMatrix(size, size);

    System.out.println("Question 1.4: " + size + "x" + size + " matrices, "
        + Runtime.getRuntime().availableProcessors() + " available processors");
    System.out.println("threads,time_ms,speedup");
    double oneThreadMs = 0; // time with 1 thread, the baseline for the speed-up
    for (int threadCount : threadCounts) {
      double averageMs = averageTimeMs(a, b, true, threadCount);
      if (threadCount == 1) {
        oneThreadMs = averageMs;
      }
      System.out.printf("%d,%.2f,%.2f%n", threadCount, averageMs, oneThreadMs /
averageMs);
    }
  }

  /**
   * Question 1.5: multiplies square matrices of increasing size both sequentially and
in
   * parallel (with the given number of threads) and prints the average time of each.
   *
   * @param threadCount number of threads for the parallel method (the best count
found in 1.4)
   */
  public static void matrixSizeExperiment(int threadCount) {
    int[] sizes = {100, 200, 500, 1000, 2000, 3000, 4000};

    System.out.println("Question 1.5: parallel method uses " + threadCount + "
threads");
    System.out.println("size,sequential_ms,parallel_ms,speedup");
    for (int size : sizes) {
      double[][] a = generateRandomMatrix(size, size);
      double[][] b = generateRandomMatrix(size, size);
      double sequentialMs = averageTimeMs(a, b, false, threadCount);
      double parallelMs = averageTimeMs(a, b, true, threadCount);
      System.out.printf("%d,%.2f,%.2f,%.2f%n", size, sequentialMs, parallelMs,
          sequentialMs / parallelMs);
```

```
    }
  }

  /**
   * Times one multiplication method: one un-timed warm-up run, then the average of
REPETITIONS
   * timed runs, in milliseconds.
   *
   * @param a is the first matrix
   * @param b is the second matrix
   * @param parallel true to time parallelMultiplyMatrix, false to time
sequentialMultiplyMatrix
   * @param threadCount number of threads (only used when parallel is true)
   * @return average execution time in milliseconds
   */
  private static double averageTimeMs(double[][] a, double[][] b, boolean parallel,
      int threadCount) {
    double totalMs = 0;
    for (int run = 0; run <= REPETITIONS; run++) { // run 0 is the warm-up
      long startTime = System.nanoTime();
      if (parallel) {
        parallelMultiplyMatrix(a, b, threadCount);
      } else {
        sequentialMultiplyMatrix(a, b);
      }
      double elapsedMs = (System.nanoTime() - startTime) / 1_000_000.0;
      if (run > 0) {
        totalMs += elapsedMs;
      }
    }
    return totalMs / REPETITIONS;
  }

  /**
   * Questions 1.1 and 1.2: checks both multiplication methods against hand-computed
results,
   * algebraic identities, expected shapes, and each other.
   *
   * @return true if every check passed
   */
  public static boolean validate() {
    boolean allPassed = true;

    // 1. Hand-computed 2x2 product
    double[][] a2 = {{1, 2}, {3, 4}};
    double[][] b2 = {{5, 6}, {7, 8}};
    double[][] expected2 = {{19, 22}, {43, 50}};
    allPassed &= check("2x2 hand-computed (sequential)",
```

```
        Arrays.deepEquals(sequentialMultiplyMatrix(a2, b2), expected2));
    allPassed &= check("2x2 hand-computed (parallel, 2 threads)",
        Arrays.deepEquals(parallelMultiplyMatrix(a2, b2, 2), expected2));

    // 2. Hand-computed non-square 2x3 * 3x2 product
    double[][] a23 = {{1, 2, 3}, {4, 5, 6}};
    double[][] b32 = {{7, 8}, {9, 10}, {11, 12}};
    double[][] expected22 = {{58, 64}, {139, 154}};
    allPassed &= check("2x3 * 3x2 hand-computed (sequential)",
        Arrays.deepEquals(sequentialMultiplyMatrix(a23, b32), expected22));
    allPassed &= check("2x3 * 3x2 hand-computed (parallel, 2 threads)",
        Arrays.deepEquals(parallelMultiplyMatrix(a23, b32, 2), expected22));

    // 3. Algebraic identities on a random 50x50 matrix: M * I = M and M * 0 = 0
    double[][] matrix = generateRandomMatrix(50, 50);
    double[][] identity = new double[50][50];
    for (int i = 0; i < 50; i++) {
      identity[i][i] = 1;
    }
    double[][] zero = new double[50][50];
    allPassed &= check("M * I = M (sequential)",
        Arrays.deepEquals(sequentialMultiplyMatrix(matrix, identity), matrix));
    allPassed &= check("M * I = M (parallel, 4 threads)",
        Arrays.deepEquals(parallelMultiplyMatrix(matrix, identity, 4), matrix));
    allPassed &= check("M * 0 = 0 (sequential)",
        Arrays.deepEquals(sequentialMultiplyMatrix(matrix, zero), zero));
    allPassed &= check("M * 0 = 0 (parallel, 4 threads)",
        Arrays.deepEquals(parallelMultiplyMatrix(matrix, zero, 4), zero));

    // 4. Shape and parallel-vs-sequential comparison on a non-square 301x200 *
200x157 product.
    // 3 and 7 do not divide 301 (uneven split); 500 > 301 tests the cap on the thread
count.
    double[][] a = generateRandomMatrix(301, 200);
    double[][] b = generateRandomMatrix(200, 157);
    double[][] sequentialResult = sequentialMultiplyMatrix(a, b);
    allPassed &= check("301x200 * 200x157 gives a 301x157 result",
        sequentialResult.length == 301 && sequentialResult[0].length == 157);
    for (int threadCount : new int[] {1, 2, 3, 7, 16, 500}) {
      allPassed &= check("parallel (" + threadCount + " threads) equals sequential",
          Arrays.deepEquals(parallelMultiplyMatrix(a, b, threadCount),
sequentialResult));
    }
    return allPassed;
  }

  /**
   * Prints the outcome of one validation check.
```

```
   *
   * @param name description of the check
   * @param passed whether the check passed
   * @return passed, so results can be accumulated
   */
  private static boolean check(String name, boolean passed) {
    System.out.println((passed ? "PASS  " : "FAIL  ") + name);
    return passed;
  }
}
package ca.mcgill.ecse420.a1;

import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;


public class Deadlock {

  // How long each thread holds its first lock before requesting the second one (ms)
  private static final long HOLD_TIME_MS = 100;
  // How long main waits for the threads before checking for deadlock (ms)
  private static final long JOIN_TIMEOUT_MS = 2000;

  private static final Lock lockA = new ReentrantLock();
  private static final Lock lockB = new ReentrantLock();

  public static void main(String[] args) throws InterruptedException {
    boolean useFix = args.length > 0 && args[0].equals("fixed");

    Thread thread1 = new Thread(() -> acquireBothLocks("Thread 1", lockA, "A", lockB,
"B"),
        "Thread 1");
    Thread thread2;
    if (useFix) {
      // Question 2.2: same global order as Thread 1 (A before B), so no circular wait
      thread2 = new Thread(() -> acquireBothLocks("Thread 2", lockA, "A", lockB, "B"),
          "Thread 2");
    } else {
      // Question 2.1: opposite order (B before A), which allows a circular wait
      thread2 = new Thread(() -> acquireBothLocks("Thread 2", lockB, "B", lockA, "A"),
          "Thread 2");
    }
    // Daemon threads do not keep the JVM alive, so the program can exit after a
deadlock
    thread1.setDaemon(true);
```

```
    thread2.setDaemon(true);

    thread1.start();
    thread2.start();
    thread1.join(JOIN_TIMEOUT_MS);
    thread2.join(JOIN_TIMEOUT_MS);

    if (!thread1.isAlive() && !thread2.isAlive()) {
      System.out.println("Both threads finished.");
    } else {
      reportDeadlock();
    }
  }

  /**
   * Acquires the first lock, waits a little, acquires the second lock, then releases
both.
   *
   * @param threadName name of the thread, used in the messages
   * @param firstLock first lock to acquire
   * @param firstName name of the first lock
   * @param secondLock second lock to acquire
   * @param secondName name of the second lock
   */
  private static void acquireBothLocks(String threadName, Lock firstLock, String
firstName,
      Lock secondLock, String secondName) {
    firstLock.lock();
    try {
      System.out.println(threadName + ": acquired lock " + firstName);
      sleep(HOLD_TIME_MS); // gives the other thread time to take its own first lock

      System.out.println(threadName + ": waiting for lock " + secondName
          + " while holding lock " + firstName);
      secondLock.lock(); // in the deadlock version this blocks forever
      try {
        System.out.println(threadName + ": acquired lock " + secondName);
      } finally {
        secondLock.unlock();
      }
    } finally {
      firstLock.unlock();
    }
  }

  /** Uses the JVM's built-in deadlock detector to print which threads are deadlocked.
*/
  private static void reportDeadlock() {
```

```
    ThreadMXBean threadBean = ManagementFactory.getThreadMXBean();
    long[] deadlockedIds = threadBean.findDeadlockedThreads();
    if (deadlockedIds == null) {
      System.out.println("Threads still running after " + JOIN_TIMEOUT_MS
          + " ms, but no deadlock was detected.");
      return;
    }
    System.out.println("DEADLOCK detected after " + JOIN_TIMEOUT_MS + " ms:");
    for (ThreadInfo info : threadBean.getThreadInfo(deadlockedIds)) {
      System.out.println("  " + info.getThreadName() + " is blocked on a lock owned by
"
          + info.getLockOwnerName());
    }
    System.out.println("\"Both threads finished.\" was never printed.");
  }

  /**
   * Sleeps for the given number of milliseconds.
   *
   * @param milliseconds time to sleep
   */
  private static void sleep(long milliseconds) {
    try {
      Thread.sleep(milliseconds);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  }
}
package ca.mcgill.ecse420.a1;

import java.lang.management.ManagementFactory;
import java.lang.management.ThreadMXBean;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;


public class DiningPhilosophers {

  private static final int MAX_ACTION_TIME_MS = 10; // longest time spent thinking or
eating
  private static final long DEADLOCK_CHECK_INTERVAL_MS = 100;

  public static void main(String[] args) throws InterruptedException {
    String mode = args.length > 0 ? args[0] : "nostarvation";
```

```
    int durationSeconds = args.length > 1 ? Integer.parseInt(args[1]) : 8;
    int numberOfPhilosophers = args.length > 2 ? Integer.parseInt(args[2]) : 5;

    boolean orderedChopsticks;
    boolean fairLocks;
    switch (mode) {
      case "deadlock":
        orderedChopsticks = false;
        fairLocks = false;
        break;
      case "nodeadlock":
        orderedChopsticks = true;
        fairLocks = false;
        break;
      case "nostarvation":
        orderedChopsticks = true;
        fairLocks = true;
        break;
      default:
        System.out.println("Unknown mode '" + mode
            + "'. Use deadlock, nodeadlock or nostarvation.");
        return;
    }
    System.out.println("Mode: " + mode + ", " + numberOfPhilosophers + " philosophers,
"
        + durationSeconds + " s, ordered chopsticks: " + orderedChopsticks + ", fair
locks: "
        + fairLocks);

    // Fair locks hand the lock to the longest-waiting thread; non-fair locks allow
barging
    ReentrantLock[] chopsticks = new ReentrantLock[numberOfPhilosophers];
    for (int i = 0; i < numberOfPhilosophers; i++) {
      chopsticks[i] = new ReentrantLock(fairLocks);
    }

    Philosopher[] philosophers = new Philosopher[numberOfPhilosophers];
    ExecutorService executor = Executors.newFixedThreadPool(numberOfPhilosophers);
    for (int i = 0; i < numberOfPhilosophers; i++) {
      int left = i;
      int right = (i + 1) % numberOfPhilosophers;
      int first = orderedChopsticks ? Math.min(left, right) : left;
      int second = orderedChopsticks ? Math.max(left, right) : right;
      philosophers[i] = new Philosopher(i, chopsticks[first], first,
chopsticks[second], second);
      executor.execute(philosophers[i]);
    }
```

```
    // Let the simulation run, checking regularly whether the philosophers are
deadlocked
    boolean deadlocked = false;
    long endTime = System.currentTimeMillis() + durationSeconds * 1000L;
    while (System.currentTimeMillis() < endTime && !deadlocked) {
      Thread.sleep(DEADLOCK_CHECK_INTERVAL_MS);
      deadlocked = isDeadlocked();
    }

    // Stop every philosopher (interrupts sleeping and waiting threads) and wait for
them to exit
    executor.shutdownNow();
    executor.awaitTermination(5, TimeUnit.SECONDS);

    System.out.println();
    if (deadlocked) {
      System.out.println("DEADLOCK detected: every philosopher holds one chopstick and
waits"
          + " for the next one.");
    } else {
      System.out.println("No deadlock occurred during the " + durationSeconds + " s
run.");
    }
    System.out.println("Meals eaten:");
    for (Philosopher philosopher : philosophers) {
      System.out.println("  Philosopher " + philosopher.getId() + ": "
          + philosopher.getMealsEaten());
    }
  }

  /**
   * Uses the JVM's built-in deadlock detector, which also understands ReentrantLock.
   *
   * @return true if some threads are deadlocked
   */
  private static boolean isDeadlocked() {
    ThreadMXBean threadBean = ManagementFactory.getThreadMXBean();
    return threadBean.findDeadlockedThreads() != null;
  }

  /** A philosopher alternates between thinking and eating; eating needs both
chopsticks. */
  public static class Philosopher implements Runnable {

    private final int id;
    private final ReentrantLock firstChopstick;
    private final int firstNumber;
    private final ReentrantLock secondChopstick;
```

```
    private final int secondNumber;
    // Written by the philosopher's thread, read by main at the end, so it must be
thread-safe
    private final AtomicInteger mealsEaten = new AtomicInteger();

    /**
     * Creates a philosopher.
     *
     * @param id philosopher number
     * @param firstChopstick chopstick picked up first
     * @param firstNumber number of the first chopstick, used in the messages
     * @param secondChopstick chopstick picked up second
     * @param secondNumber number of the second chopstick, used in the messages
     */
    public Philosopher(int id, ReentrantLock firstChopstick, int firstNumber,
        ReentrantLock secondChopstick, int secondNumber) {
      this.id = id;
      this.firstChopstick = firstChopstick;
      this.firstNumber = firstNumber;
      this.secondChopstick = secondChopstick;
      this.secondNumber = secondNumber;
    }

    @Override
    public void run() {
      try {
        while (!Thread.currentThread().isInterrupted()) {
          doAction("is thinking");

          // lockInterruptibly (instead of lock) lets main stop a philosopher that is
waiting
          firstChopstick.lockInterruptibly();
          try {
            log("picked up chopstick " + firstNumber);

            secondChopstick.lockInterruptibly();
            try {
              log("picked up chopstick " + secondNumber);
              int meal = mealsEaten.incrementAndGet();
              doAction("is eating (meal " + meal + ")");
            } finally {
              secondChopstick.unlock();
              log("put down chopstick " + secondNumber);
            }
          } finally {
            firstChopstick.unlock();
            log("put down chopstick " + firstNumber);
          }
```

```
        }
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // end of the simulation
      }
    }

    public int getId() {
      return id;
    }

    public int getMealsEaten() {
      return mealsEaten.get();
    }

    /**
     * Prints the action, then spends a random time (0 to MAX_ACTION_TIME_MS) doing
it.
     *
     * @param action description of the action
     * @throws InterruptedException if the simulation is stopped
     */
    private void doAction(String action) throws InterruptedException {
      log(action);
      Thread.sleep((long) (Math.random() * MAX_ACTION_TIME_MS));
    }

    /**
     * Prints a message prefixed with the philosopher's number.
     *
     * @param message text to print
     */
    private void log(String message) {
      System.out.println("Philosopher " + id + " " + message);
    }
  }
}
```

Appendix B – AI Use Disclosure

[Draft — review and edit so it describes exactly what you did. Add any other AI tools you used.]

AI tool(s) used: Chat gpt

Nature of use: reviewing our report and suggesting code change for renaming variables to follow the Google Java Style Guide and separate modes for the deadlocking, deadlock-free and starvation-free versions of the dining philosophers); restructuring the report

Verification activities: we read all the AI-suggested modification by line to make sure we understand it. We compiled and ran every program on our own machine (Apple M1 Pro): the `validate` mode compares the parallel result with the sequential result and with hand-computed products, the deadlock programs were checked to deadlock (and the fixed versions to finish), and the meal counts were checked for starvation. All timing results in the report were measured on our machine, not produced by the AI. The Question 4 formulas were checked by substituting numbers (e.g. n = 4, k = 12 exactly doubles the speed-up in 4.2).

Reflection:

- *Useful contribution:* it found inconsistencies we had missed: the report described a `validate` mode and variable names that did not exist in our code, and the Q2.1 sample output did not match what the program prints.
- *Limitation / error:* The handout says the sequential time *percentage* is decreased, with the sequential run still taking unit time, and under that wording our original answers are correct. It also first pasted example outputs from its own 2-core test environment instead of our machine.
- *How it was resolved:* we gave it the exact wording of Question 4, re-derived both answers under that wording and checked them numerically, kept our original answers and added the validity conditions. We replaced all program outputs with outputs from our own runs.